

SFWRENG 3A04

Complete Lecture Notes

Matthew Farah

March 2, 2026

Contents

1	Introduction	2
2	SKIPPED	3
3	SKIPPED	4
4	Requirements	5
5	Environments	6
6	Viewpoints	7
7	Modules	8
8	Exercises in Understanding Design Basics	9
9	Basics in Software Architecture	10
10	SKIPPED	11
11	Connectors	12
12	The 4 + 1 Model	13
13	Patterns	14
14	Modelling Software Systems	15
15	Interaction-Oriented Architectures	17
16	SKIPPED	19
17	SKIPPED	20
18	SKIPPED	21
19	Exercises in Data-Oriented Architectures	22

Lecture 1

Introduction

- There are two types of design:
 1. Functional design.
 2. Non-functional design.
- A design principle is a rule of thumb.
 - Software architectures are critiqued according to some given design principle(s).

Lecture 2

SKIPPED

- SKIPPED

Lecture 3

SKIPPED

- SKIPPED

Lecture 4

Requirements

- Design is best defined as the process of embedding some function into a form.
- The form a design takes should be:
 1. Dependable.
 2. Sustainable.
 3. 'Harmonious' with its environment
- Functional requirements describe the functionality of the design.
 - Describes the design's reason for existence.
- Non-functional requirements describe the form that design should take.
 - Describes how the design should be manifested or instantiated
- Non-functional requirements can put specific requirements on the design's speed, security, ease-of-use, and so on.
- The environment and functional requirements of a design place constraints on its implementation.

Lecture 5

Environments

- Systems are described by their design.
 - Functional requirements provide the reason for the design's existence.
- Systems always exist in some given environment.
 - Non-functional requirements describe how the design should work with its environment.
- Systems and their environments are always interacting in some way.
 - The environment always initiates these interactions.
- A 'scenario' is a sequence of interactions between a system and its environment.
 - A global scenario is the integration of all scenarios together for some given system and environment pair.
- A 'business event' is something a system must perform when prompted by its environment.
 - Scenarios terminate when a system fulfills the business task it was prompted to perform by its environment.
- A 'business process' is the act systems engage in to fulfil business events.
 - One or more components of a system may be engaged in any given business process.
- A 'viewpoint' is a perspective that designs should take into account.
 - For example, a legal viewpoint, an economic viewpoint, etc.
 - Viewpoints are given for each business event.
 - * Inapplicable viewpoints should still be directly mentioned and specified as so.
- The modes of a system describe equivalence classes of similar system behaviours.

Lecture 6

Viewpoints

- When considering a written, non-technical description of a system:
 - Verbs often describe the system's functions.
 - Nouns often describe what the system's functions interacts with.
- The different modes of a system represent an equivalence class of the system's state.
 - A system's state describes its behaviour in some way.
 - A system's mode changes in response to business events.
- All viewpoints contribute to a system's global scenario.
- Designs should ideally be open for addition but closed for modification.

Lecture 7

Modules

I was really tired this lecture so I didn't take many notes, sorry.

- Systems are composed of modules.
 - Modules are intended to address a single concern.
 - * Modules act as an abstraction over how the system deals with a concern.
 - In this sense, modules hide information about the system.
 - A modules 'secret' is that which a module hides.
 - * A single concern may be shared across multiple business events.
 - * Modules have minimal knowledge of each other.
- There are three types of modules:
 1. Behaviour hiding modules.
 2. Decision hiding modules.
 3. Interface hiding modules.

Lecture 8

Exercises in Understanding Design Basics

I didn't take notes for each example taken up in class for this one, just information that seemed generally relevant

- Choosing the form that a system should take is more challenging than what functionalities it should have.
- A system is a collection of one or more elements in an environment which fulfills one or more tasks.
 - Composite systems are defined as systems which use more than one such element.
- Modules are used to hide secrets.
- Business events are independent of one another.

Lecture 9

Basics in Software Architecture

- An architect is a person responsible for formulating the design of a system.
- An architect must always provide three things when designing a system for the design to be valid:
 1. The system's elements.
 2. The system's connectors.
 3. The rationale behind the elements/connectors chosen.
- The component parts of a system, particularly its modules, should have minimal knowledge about one another.
- Structure graphs can have connectors with the following attributes:
 1. Direction.
 2. Synchronization.
 3. Sequence.
- Connectors at a given level of a structure graph inherit attributes from their code-structure counterparts.
 - For example, their multiplicity, their distance, and the medium over which the connection is performed.

Lecture 10

SKIPPED

- SKIPPED.

Lecture 11

Connectors

- Connector fan-out describes how many elements a given connector connects.
- Connectors can exist in one of two different kinds of environments:
 1. Homogeneous environments.
 - An environment is homogeneous when all elements are using the same programming language/framework.
 2. Heterogeneous environments.
 - An environment is heterogeneous if it is not homogeneous.
- The ‘4 + 1’ model is used to provide an overview on different aspects of the system.
 - Composed of the following 5 categories:

Lecture 12

The 4 + 1 Model

- The 4 + 1 model is composed of 5 distinct views.
 - Each view can be expressed with models reflecting their design.
- Below is a list of some acceptable models each view in the 4 + 1 model can use to express their design.
 1. The logical view:
 - Class diagrams.
 - Block diagrams.
 - Sequence diagrams.
 - Communication diagrams.
 2. The process view:
 - Activity diagrams.
 - Interaction diagrams.
 3. The development view:
 - Package diagrams.
 4. The physical view:
 - Deployment diagrams.
- ‘Patterns’ are reliable solutions for frequently encountered problems in design.
- ‘Architectural styles’ are equivalence classes describing systems which use similar patterns when solving a given problem.
- In use case diagrams:
 - ‘Use case controllers’ orchestrate what use case (or mode) the system is in.
 - * Use case controllers can be represented mathematically as Finite State Machines (FSMs)
- In class diagrams:
 - Boundary classes are interface hiding classes.

Lecture 13

Patterns

- Generally, patterns are regular occurrences of an object in some given space/dimension.
 - ‘Architectural patterns’ are solutions to problems belonging to some family of other, similar, architectural patterns.
- An ‘architectural style’ is a set of characteristics and features which distinguish architectural designs from one another.
- The repeated use of certain patterns defines a software’s architectural style.
 - A given piece of software may be composed of multiple different architectural styles.
- An architecture’s ‘orientation’ the kinds of concerns it is focused on addressing.
- ‘Interaction-oriented’ software architectures focus on challenges associated with:
 1. Components interacting with other components.
 2. Components interacting with the environment.
- Some examples of interaction-oriented software include PAC and MVC.
 - Both will be explained in detail in later lectures.
- Interaction-oriented architectures segregate a system’s components into three major modules:
 - A data module.
 - A flow control module.
 - A view/presentation module.
- Systems with high coupling generally:
 - Make modifying the system more difficult.
 - Are better at separating out concerns.

Lecture 14

Modelling Software Systems

- Models of a software system's design are:
 1. Abstract representations of knowledge of the system.
 2. An approximation of some aspects of the system.
 3. Can be expressed using graphs, natural language, math, and other ways.
- A system's design models are used to:
 1. Communicate knowledge about the system.
 2. Help articulate and assess design decisions for the system by abstracting away certain details.
 3. Act as documentation to aid in maintaining the system.
- Design models can be 'formal' or 'rigorous'.
 - Don't let the word rigorous confuse you! Rigorous design models are subject to interpretability.
- Examples of formal design models include:
 1. FSMs.
 2. Graphs.
 3. Relations.
- Examples of rigorous design include:
 1. UML diagrams.
 2. Proprietary/non-standard models.
- Analysis class diagrams are an example of a rigorous design model.
- Analysis class diagrams are composed of three types of classes:
 1. Boundary classes.
 - Are interfaces between the system and its environment
 - Keep the format of communication between the system and its environment as its secret.
 2. Entity classes.
 - Store data.
 - Exchange data between classes.
 - Keep an n -tuple (representing the data the entity class stores) as its secret.
 3. Control classes.
 - Coordinate interactions between other classes in response to a business event.
- The 4 + 1 model is another example of design modelling.
 - The scenario view of the system acts as the basis for its logical view.
 - * The logical view represents the system as an analysis class diagram.
 - All other views are derived from the logical view.

- All software architectures can be said to be composed of the following parts:
 1. Elements.
 2. Connectors.
 3. The rationale behind those elements/connectors.
- Connectors always have some source and destination element.
- ‘Multiplicity’ is an attribute of connectors.
 - The multiplicity of a connector describes how many elements are related together by the connector from source to destination.
 - * The multiplicity of any connector is one of:
 1. 1-1.
 2. 1-many.
 3. Many-1.
 4. Many-many.
- ‘Distance’ is another attribute of a connector.
 - The distance of a connector describes how ‘far apart’ the elements it connects are.
 - * How far apart elements are can be categorized as:
 1. Elements belonging to the same thread.
 2. Elements belonging to the same process.
 3. Elements belonging to the same computer.
 4. Etc.

Lecture 15

Interaction-Oriented Architectures

- The two major interaction-oriented architectures covered are:
 1. Presentation, Abstraction, Control (PAC).
 2. Model, View, Control (MVC).
 - MVC actually has two closely related variants we will also cover:
 - (a) MVC-I.
 - (b) MVC-II.
- PAC and MVC are closely related.
 - The presentation module is analogous to the view module.
 - The abstraction module is analogous to the model module.
- PAC and MVC are differentiated by their overall structure.
 - MVC is composed of a single model, view, and controller element.
 - PAC is composed of a hierarchy of presentation, abstraction, and control elements.
 - * An ‘agent’ is a group of these three elements in the hierarchy working together.
- PAC agents only communicate with one another by their control elements.
- A new agent is required for each distinct source of data handled by the system.
 - Data sources are distinct if they describe different quantities or are sent to the system at different frequencies.
- There are benefits and drawbacks to using a PAC architecture.
 - Benefits:
 1. Comparatively better at handling multiple, different kinds of data.
 2. Comparatively better at handling data being received at varying frequencies.
 - Drawbacks:
 1. If a PAC hierarchy is deep, the top-level controller takes comparatively long to communicate with the bottom-level controller.
 2. If the PAC hierarchy is shallow, the top-level controller has high coupling.
- An effective implementation of a PAC architecture involves properly balancing the trade-offs associated with different hierarchical organizations of agents.
- MVC is primarily used for simpler, web-based systems.
 - MVC emphasizes the system’s look-and-feel.
- Certain software principles may be occasionally ignored if violating it is more advantageous than adhering.
- MVC-II is the usual variant of MVC discussed above.
- MVC-I is a simpler variant of MVC composed of two elements:

1. A model module.
 - Forms the system's 'back-end'.
 2. A single view/control module.
 - Forms the system's 'front-end'.
- MVC-I violates the separation of concerns principle.
 - For simple systems with a minimal need for a separate controller module, this is worth it.
 - The subscribe/notify pattern is used to implement communication between elements in MVC modules.
 - In the subscribe/notify patterns, elements can be either:
 1. Subjects.
 2. Observers.
 - Observers can subscribe to subjects to be notified of state changes.
 - Observers can unsubscribe to subjects to stop receiving such notifications.
 - Subjects track all subscribed observers.
 - Subjects have a special method to notify all subscribed observers.
 - In the MVC-I architecture:
 - The back-end is the subject.
 - The front-end is the observer.
 - In the MVC-II architecture:
 - The model element is the subject.
 - The controller and view elements are both observers of the model element.

Lecture 16

SKIPPED

- SKIPPED

Lecture 17

SKIPPED

- SKIPPED

Lecture 18

SKIPPED

- SKIPPED

Lecture 19

Exercises in Data-Oriented Architectures

- There will be 12 questions on the midterm.
- Out of 20 marks plus 3 bonus marks.
- Questions similar to end of week exercises.

Example 1: We would like to develop a software system that recognizes connected speech in a 1000-word vocabulary with correct interpretations for 90% of test sentences. Its basic methodology involves the application of symbolic reasoning as an aid to signal processing. A marriage of general artificial intelligence techniques with specific acoustic and linguistic knowledge is needed to accomplish satisfactory speech-understanding performance. Because the various techniques and heuristics employed were embedded within a general problem-solving framework, the system has to embody several design characteristics that are adaptable to other domains as well. You need to keep in mind the characteristics of the speech recognition problem in particular, the special kinds of problem-solving uncertainty in that domain. Uncertainty arises in a problem-solving system if the system's knowledge is inadequate to produce a solution directly. The fundamental method for handling uncertainty is to create a space of candidate solutions and search that space for a solution. In a difficult problem, i.e. one with a large search space, a problem solver can be effective only if it can search efficiently. To do so, it must apply knowledge to guide the search so that relatively few points in the space need be examined before a solution is found. A key way of accomplishing this is by augmenting the space of candidate solutions with candidate partial solutions and then constructing a complete solution by extending and combining partial candidates. A candidate partial solution represents all complete candidates that contain it. By considering partial solution candidates, we can often eliminate whole subspaces from further consideration and simultaneously focus the search on more promising subspaces. Also, the necessity for diverse candidate partial solution derives from the diversity of transformations used by the speaker in creating the acoustic signal and the corresponding inverse transformations needed by the listener for interpreting it.

What is the most suitable software architecture for this speech recognition system? Explain why the proposed architecture is the most suitable one.

Solution:

Notice that:

- The problem has an indeterminate solution.
- The system is composed of multiple agents:
 1. One for linguistics/acoustics.
 2. One of interpretation.
 3. Etc.
- No one agent can solve the problem on its own.
 - Hence, agents must communicate

Using the above, we can surmise that this system's design would be best served by a blackboard architecture.

Example 2: We would like to build a software system that controls a robot. It has the following typical software functions:

- Acquiring and interpreting input provided by sensor.

- Controlling the motion of wheels and other movable parts.
- Planning future paths.

There are some situations that bring challenges in designing the software system. Among them, we list the following:

- Obstacles may block path.
- Sensor input may be imperfect.
- Robot may run out of power.
- Mechanical limitations may restrict accuracy of movement.
- Unpredictable events may demand a rapid (autonomous) response.

We will use the following evaluation criteria for a proposed architecture:

1. We aim at having a robot that coordinates actions to achieve assigned objectives with the reactions imposed by the environment.
2. The robot must function in the context of incomplete, unreliable and contradictory information.
3. The robot must be robust against sensor faults and problems like reduced power supply. For example, when power is low it should be able to easily disable some of its architectural elements while carrying on with assigned work.
4. The robot has to be flexible and capable of providing support for experimentation and reconfiguration.

Propose an appropriate overall architecture. Evaluate your architecture solution against the above evaluation criteria (i.e., explain how your solution satisfies the criteria 1 to 4 listed above).

Solution:

Notice that:

- The system has many stated limitations.
- The system's evaluation criteria are all given.
- The system is also composed of multiple agents.
 1. A sensing agent.
 2. An agent to control the actuators.
 3. A navigational agent.
 4. Etc.
- The solution to the problem indeterminate.

Hence, from the above, we can surmise that the system would be best served by a blackboard architecture, potentially incorporating some Master/Slave elements for fault tolerance.

Example 3: Traditional biometric systems rely on a single biometric identifier such as fingerprint or face. A multi-biometric system integrates many biometric identifiers (such as face, fingerprint, iris, hand geometry, gait, signature, etc.) and takes advantage of the capabilities of each biometric to provide even greater performance and higher reliability. Note that each biometric identifier has its strengths and weaknesses. The choice of a particular biometric identifier typically depends on the requirements of an application.

Propose an overall architecture for a multi-biometric system and give the rationale behind your design proposal.

Solution:

Notice that:

- There are no views, so the system must use a data-oriented architecture.
- There are multiple specialized agents.
- No one agent can solve the problem on its own.
 - So collaboration, and hence communication, between agents is necessary.
- The solution to the problem is indeterminate.

Hence, from the above, we can surmise that the system would be best served by a blackboard architecture.